

TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 8: Motion Planning and Decision Making

TURTLE Incubator - SO-ARM Software Stack

Agenda

Motion Planning Overview

- What planning is and where it fits in the robotics stack

Search Algorithms

- BFS, Dijkstra, and A* - how they work and how they differ

Sampling-Based Planning

- RRT and RRT* - planning in high-dimensional joint space

Trajectory Generation

- Position profiles, velocity profiles, and smooth motion

Collision Avoidance

- Obstacles, safety margins, and joint limit checking

SO-ARM Application

- Move around obstacles and generate efficient trajectories

Demonstration + Hands-On + Deliverable

- Build a path planning algorithm and execute collision-free motion

What You Are Building Today

By end of session your script will:

- 1 Represent the SO-ARM workspace as a 2D grid with an obstacle
- 2 Run A* to find a collision-free path from start to goal
- 3 Convert the grid path to joint-space waypoints using IK
- 4 Generate a smooth trapezoidal velocity profile between waypoints
- 5 Validate every waypoint against joint limits
- 6 Execute the path on the real SO-ARM with the Week 6 PID controller
- 7 Verify the arm reaches the goal without hitting the obstacle

This connects every week into one pipeline: Weeks 3+4 (kinematics), 6 (control), 7 (filtering), 8 (planning).

Motion Planning Overview

What Is Planning and Where It Fits

Planning answers:

- Given a start and goal, what sequence of configurations gets the arm there safely?

Without planning:

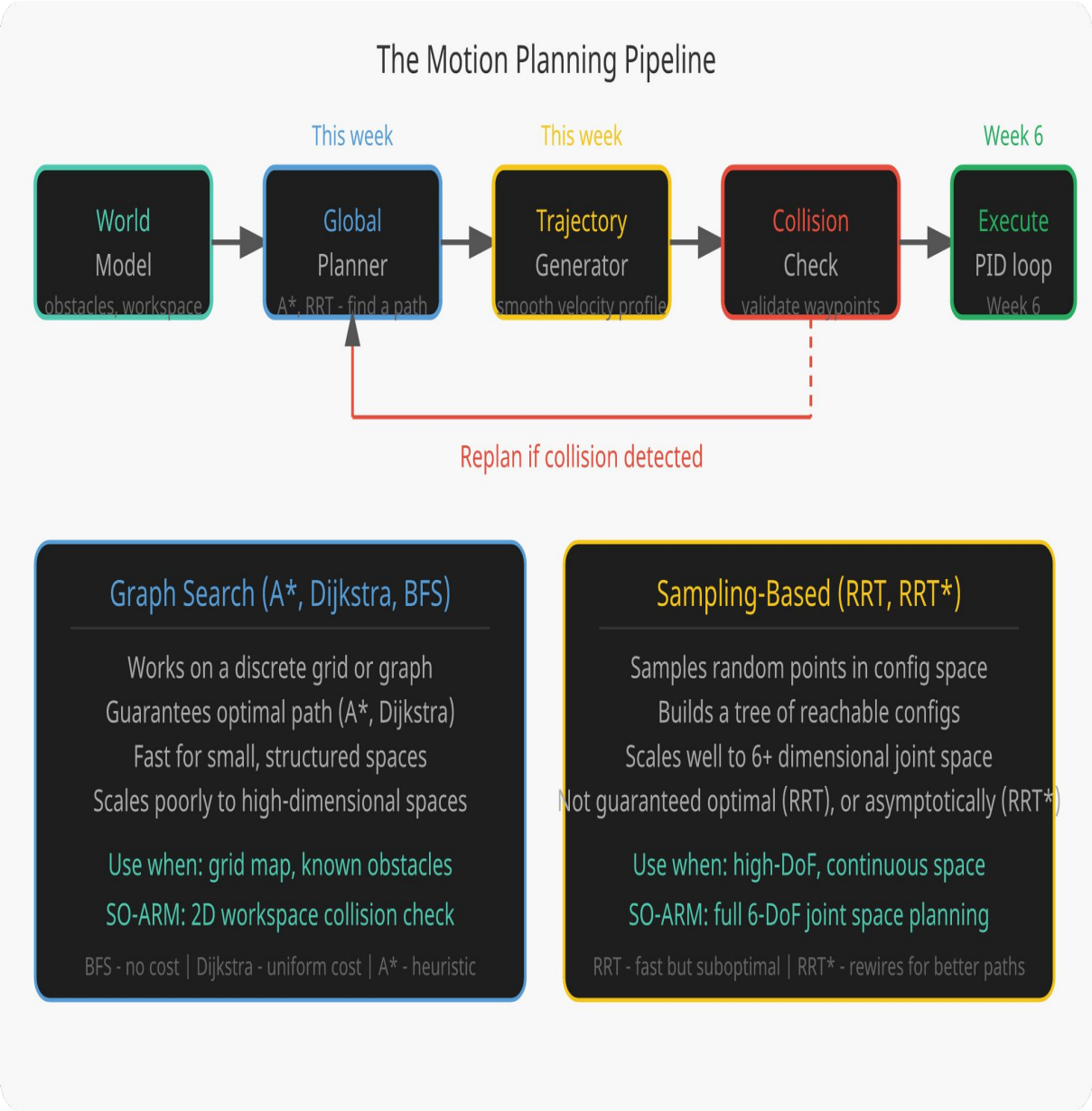
- Sending a single IK solution moves the arm in a straight joint-space line
- That line may pass through obstacles, joint limits, or singularities

With planning:

- Planner finds a sequence of waypoints that avoids all obstacles
- Trajectory generator smooths the path so motors are not shocked
- Controller (PID) follows each waypoint

Two algorithm families:

- Graph search: A*, Dijkstra, BFS
- Sampling-based: RRT, RRT*



Search Algorithms

BFS, Dijkstra, and A* - How They Differ

All three search a graph of states to find a path from start to goal:

BFS (Breadth-First Search)

- Explores all neighbors equally, layer by layer
- Finds the shortest path in hops - ignores edge costs entirely
- Simple but ignores the real cost of movement

Dijkstra's Algorithm

- Like BFS but uses a priority queue sorted by cumulative cost g(n)
- Guarantees the optimal (lowest cost) path when all edge costs are non-negative
- Explores in all directions equally - slow if goal is far away

A* (A-Star)

- Dijkstra + a heuristic h(n) that estimates the cost to goal
- `f(n) = g(n) + h(n)  |  expands the node with the lowest f first`
- Finds the optimal path and explores far fewer nodes than Dijkstra
- Heuristic must be admissible (never overestimate) - Euclidean distance works for grids

A* on a Grid - Visualized

A* finds the shortest path around obstacles:

Green S: start (0,0)
Red G: goal (9,9)
Dark gray: obstacles
Blue dots: A* path

f(n) = g(n) + h(n):

- g(n) - total cost from start to node n
- h(n) - Euclidean distance to goal (heuristic)
- A* always expands the lowest f(n) next

Key property:

- With an admissible heuristic, A* always finds the optimal path
- Explores far fewer nodes than Dijkstra on the same grid



Sampling-Based Planning

RRT - Rapidly-Exploring Random Tree

A* works on grids. For 6-DoF joint space, we use RRT:

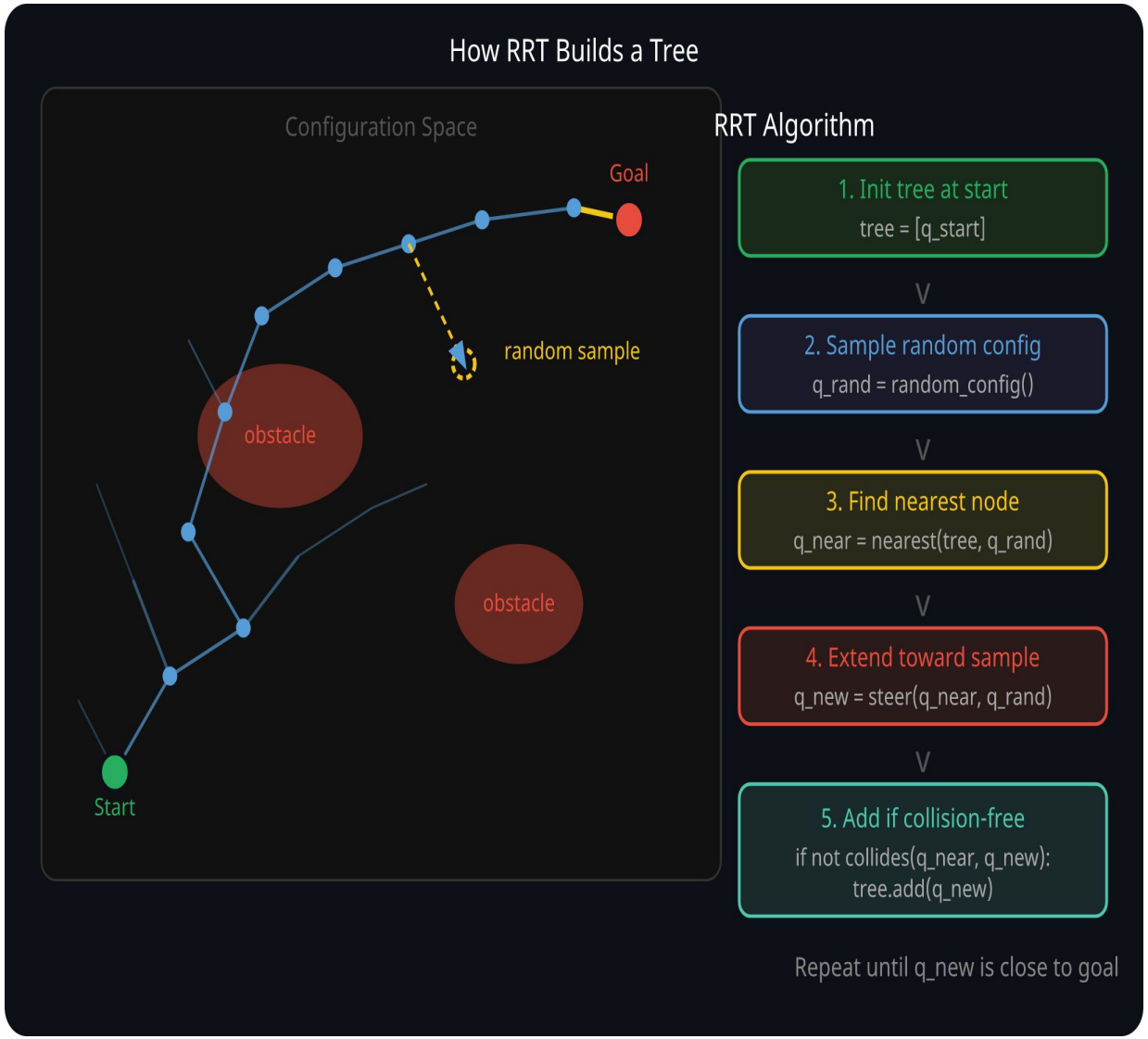
Why not A* for the SO-ARM?

- 6 joints with discrete resolution would need billions of grid cells
- RRT works directly in continuous configuration space

How RRT grows a tree:

- 1 Init tree at start configuration
- 2 Sample a random config q_rand in joint space
- 3 Find the nearest existing node q_near in the tree
- 4 Extend from q_near toward q_rand by a step size
- 5 If the new config is collision-free, add it to the tree
- 6 Repeat until a node is close enough to the goal

RRT* adds a rewire step that improves path quality over time.



RRT vs RRT* - When to Use Each

Choose based on what matters more: speed or path quality.

RRT

- Fast - finds any valid path quickly without rewiring
- Path is often jagged and suboptimal - needs post-smoothing
- Use when: real-time replanning, interactive manipulation

RRT*

- Slower - rewires the tree after each node is added
- Asymptotically optimal - path quality improves with more iterations
- Use when: offline planning where quality matters

For the SO-ARM today:

- We implement a simplified grid-based A* in 2D workspace (fast, easy to visualize)
- Then convert the 2D waypoints to 3D Cartesian coords and use IK to get joint angles
- This avoids the complexity of full 6D RRT while covering all the key concepts

Trajectory Generation

Smooth Motion - Velocity Profiles

A path is a sequence of waypoints.

A trajectory adds timing - how fast to move between them.

Instant jump (no profile)

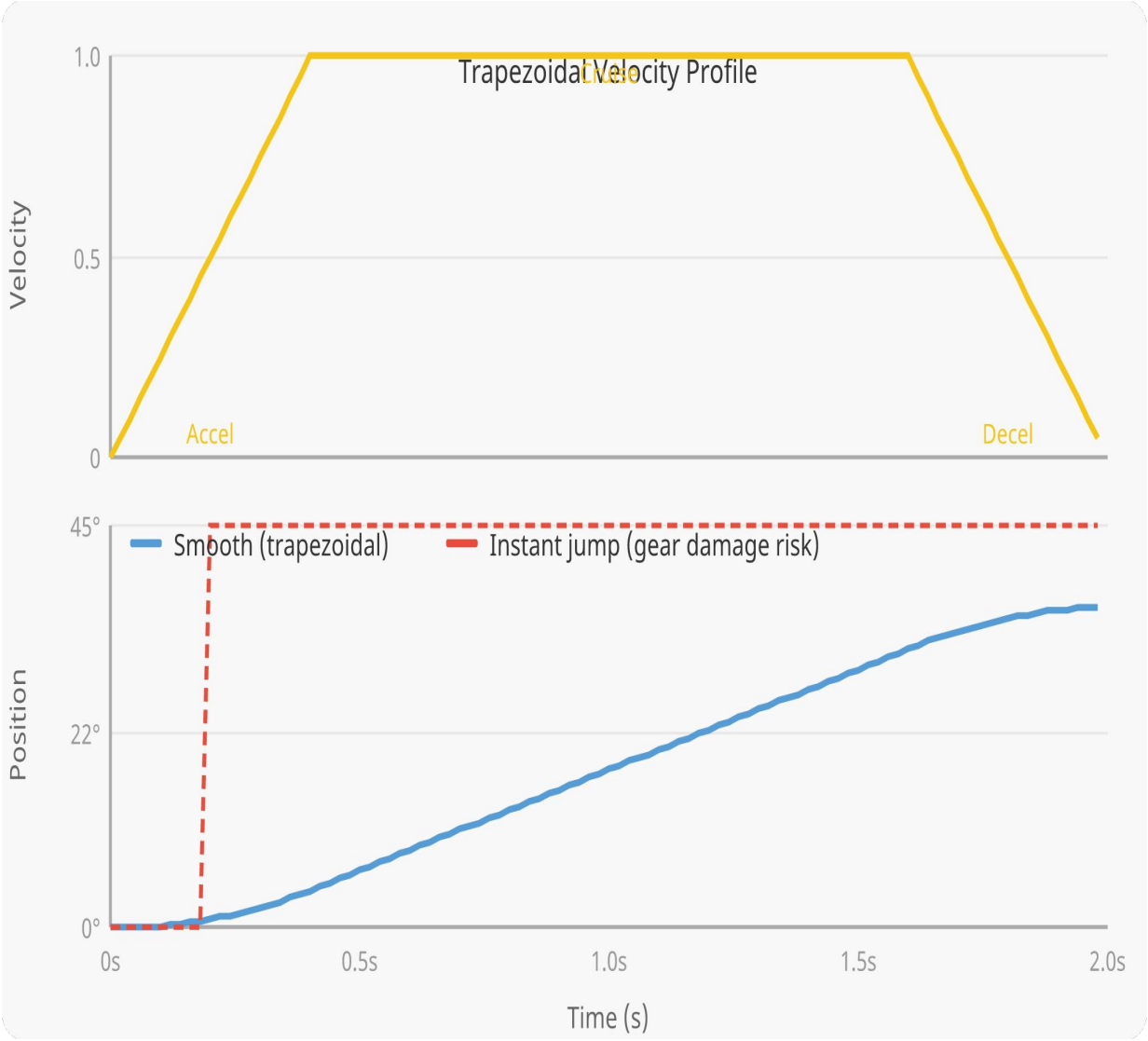
- Motor goes from 0 to full speed instantly
- Infinite acceleration - destroys gears and strips servos

Trapezoidal profile (recommended)

- Accelerate to cruise speed, cruise, then decelerate
- Bounded acceleration - smooth and hardware-safe

S-curve profile (advanced)

- Smooth the jerk (rate of acceleration change) as well
- Used in industrial arms where vibration must be minimized
- Not needed for the SO-ARM today



Position and Velocity Profile Math

Trapezoidal profile parameters:

- `v_max   - peak velocity (degrees/second)`
- `a_max   - acceleration (degrees/second^2)`
- `d_total - total angular distance to travel`

Three phases:

- `Accelerate: v(t) = a_max * t                until v = v_max`
- `Cruise:     v(t) = v_max                    until decel begins`
- `Decelerate: v(t) = v_max - a_max * (t-t2)   until v = 0`

Position from velocity:

- `theta(t) = theta_start + integral of v(t) dt`

Safety check: if d_total is too small for a full cruise phase, use triangular profile (no cruise).

Collision Avoidance

Collision Avoidance and Safety Margins

Every waypoint must be validated before execution:

Collision checking

- For each waypoint, check if any arm link intersects an obstacle
- Use bounding boxes or sphere approximations for speed

Safety margins

- Inflate obstacle bounding boxes before planning
- Planner never gets closer than margin - allows for tracking error

Joint limit checking

- Every waypoint must satisfy all 6 joint limits from so100.yaml

SO-ARM lab setup:

- Obstacles = desk objects defined by (x,y,radius) in workspace coordinates
- Safety margin = 50 mm around each object
- Check all waypoints before sending any motion command

SO-ARM Application

SO-ARM Motion Planning Setup

How we represent the planning problem for the SO-ARM:

Workspace representation

- 2D grid in the (x, z) plane - forward reach vs height above desk
- Each cell is 10mm x 10mm - 30x30 grid covers most of the arm's reach
- Obstacle cells inflated by 50mm safety margin before A* runs

Waypoint conversion

- A* outputs (row, col) grid coordinates
- Convert to (x, y=0, z) Cartesian coordinates
- Run IK on each waypoint to get joint angles
- Validate joint angles against limits before adding to the trajectory

Execution

- Trapezoidal profile generates intermediate joint angles between each pair of waypoints
- Week 6 PID controller tracks each intermediate point at 50 Hz

A* Implementation

```
import heapq, numpy as np

def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    def h(a, b): return ((a[0]-b[0])**2 + (a[1]-b[1])**2)**0.5  # Euclidean heuristic

    open_set = [(h(start,goal), 0, start, [start])]  # (f, g, node, path)
    visited  = set()

    while open_set:
        f, g, current, path = heapq.heappop(open_set)
        if current == goal:  return path              # found it
        if current in visited: continue
        visited.add(current)

        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)]:
            nr, nc = current[0]+dr, current[1]+dc
            if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==0:
                g_new = g + (1.414 if dr and dc else 1.0)
                heapq.heappush(open_set, (g_new+h((nr,nc),goal), g_new, (nr,nc), path+[(nr,nc)]))
    return None  # no path found
```

Convert Grid Path to Joint Waypoints

```
# Grid to Cartesian (x, y=0, z) in meters
CELL_SIZE = 0.01   # 10mm per cell
ORIGIN_X  = 0.05   # workspace starts 5cm forward from base
ORIGIN_Z  = 0.02   # workspace starts 2cm above desk

def grid_to_cartesian(row, col):
    x = ORIGIN_X + col * CELL_SIZE
    z = ORIGIN_Z + (grid_rows - row) * CELL_SIZE  # flip row: top of grid = higher z
    return [x, 0.0, z]

# Convert A* path to joint-space waypoints using IK
def path_to_waypoints(path, chain, current_joints):
    waypoints = []
    prev = [0] + list(current_joints) + [0]
    for row, col in path:
        target = grid_to_cartesian(row, col)
        sol    = chain.inverse_kinematics(target, initial_position=prev)
        angles = np.rad2deg(sol[1:-1])
        if validate_limits(angles):              # from Week 4
            waypoints.append(angles)
            prev = sol
    return waypoints
```

Trapezoidal Velocity Profile and Execution

```
def trapezoid_profile(q_start, q_end, v_max=30, a_max=60, dt=0.02):
    d     = np.abs(q_end - q_start)           # distance for each joint
    t_acc = v_max / a_max                     # time to reach cruise speed
    d_acc = 0.5 * a_max * t_acc**2            # distance covered in accel phase

    if np.any(d < 2 * d_acc):                 # triangular profile if distance is short
        t_acc = np.sqrt(d.max() / a_max)
        t_total = 2 * t_acc
    else:
        t_cruise = (d.max() - 2*d_acc) / v_max
        t_total  = 2*t_acc + t_cruise

    times  = np.arange(0, t_total, dt)
    interp = np.linspace(0, 1, len(times))   # normalized time
    # Smooth interpolation using cosine ease
    smooth = (1 - np.cos(np.pi * interp)) / 2
    return [q_start + smooth[i]*(q_end-q_start) for i in range(len(times))]
```

Executing the Full Planned Path

```
# Full pipeline: plan, generate trajectory, execute
grid = build_occupancy_grid(obstacles, safety_margin=0.05)  # inflate obstacles
path = astar(grid, start_cell, goal_cell)

if path is None:
    print('No valid path found - check obstacle positions or expand workspace')
else:
    waypoints = path_to_waypoints(path, chain, robot.read_all_joints())
    print(f'Plan: {len(waypoints)} waypoints, executing...')

    try:
        for i in range(len(waypoints) - 1):
            segment = trapezoid_profile(waypoints[i], waypoints[i+1])
            for q_target in segment:                    # each intermediate point
                action = dict(zip(robot.motor_names[:5], q_target))
                robot.send_action(action)               # PID tracks this
                time.sleep(0.02)                        # 50 Hz
    finally:
        robot.bus.write('Goal_Torque', [0]*5, robot.motor_names[:5])  # safe stop
```

Demonstration

Demonstration - Planned vs Unplanned Movement

Watch carefully - you will reproduce this in the hands-on.

What we will show:

- 1. Place an obstacle (box) in the SO-ARM workspace
- 2. Send a naive IK command to the goal - arm moves in a straight joint-space line and hits the box
- 3. Build the occupancy grid and visualize it with matplotlib
- 4. Run A* and visualize the path on the grid
- 5. Convert path to Cartesian waypoints and run IK - show all waypoints printed
- 6. Generate the trapezoidal velocity profile and plot it
- 7. Execute the planned path - arm moves smoothly around the obstacle
- 8. Show what happens if you request an unreachable goal (no path found)

Hands-On

Hands-On - Step 1: Build the Grid and Run A*

Goal: define a workspace grid, place an obstacle, and find a path with A*.

1. **Build a 30x30 occupancy grid (0=free, 1=obstacle)**
- `grid = np.zeros((30, 30), dtype=int)`
- `grid[10:20, 12:16] = 1   # obstacle block`

2. **Run A* from (0,0) to (29,29)**
- `path = astar(grid, (0,0), (29,29))`
- `print(f'Path found: {len(path)} waypoints')`

3. **Visualize with matplotlib**
- plt.imshow(grid, cmap='gray_r') - plot the grid
- Mark start (green), goal (red), and path (blue dots)

4. **Try blocking all paths and verify A* returns None gracefully**

Hands-On - Step 2: Execute Collision-Free Motion on the SO-ARM

Goal: convert the A* path to joint waypoints and move the arm around the obstacle.

1. **Convert the grid path to Cartesian coordinates**
   - Use grid_to_cartesian() for each (row, col) in the path

2. **Run IK on each Cartesian waypoint**
   - Validate each solution against joint limits - skip invalid waypoints

3. **Generate trapezoidal profiles between consecutive waypoints**
   - Use v_max=20 deg/s and a_max=40 deg/s^2 as starting values

4. **Execute and verify**
   - Arm should move smoothly from start to goal without hitting the obstacle
   - Log actual joint positions at 50 Hz and plot the trajectory

5. **Move the obstacle and re-run - verify A* finds a new path automatically**

Common Mistakes

3 Errors You Will Definitely Hit

Every student hits these. Read this before starting.

Error 1 - A* returns None but the path exists

- Obstacle inflation is too aggressive - the inflated obstacles block all routes
- Reduce safety_margin or increase grid resolution and re-run
- Always visualize the inflated grid before running A* - see what A* actually sees

Error 2 - IK fails on some waypoints along the path

- A* path passes through Cartesian positions that are outside the SO-ARM workspace
- Constrain A* start and goal to the IK-reachable region (x in [0.05,0.25], z in [0.05,0.30])
- Add IK failure handling: skip waypoint, warn, and continue to next

Error 3 - Arm jerks between waypoints

- Waypoints are too far apart - the trapezoidal profile spans too large an angle
- Reduce A* grid cell size to 5mm or interpolate extra waypoints between A* nodes
- Reduce v_max and a_max if the arm is shaking at transitions

Deliverable

Deliverable - Generate Collision-Free Arm Motion

Demonstrate all four checkpoints to a lab lead before leaving:

1 A* running and path visualized

- Occupancy grid shown in matplotlib with start, goal, obstacle, and planned path marked

2 Waypoints converted and validated

- IK solutions for all waypoints printed. Joint limit check passes without errors.

3 Arm executes the collision-free path

- Arm moves smoothly from start to goal position. No contact with the obstacle.

4 Replanning demonstrated

- Move the obstacle to a new position, re-run A*, and show the arm takes a different route.